

K-디지털 트레이닝

Json-server 연동 Diary 앱

[KB] IT's Your Life

✓ 일기장 프로젝트 생성

- `npm init vue diary-app2`
 - 라이브러리 선택
 - Vue-Router
 - Pinia
- 추가 라이브러리
 - `npm i bootstrap@5 axios`
- 일기장 `db.json` 파일 준비
 - `json-server db.json` 실행

Gemini.md

Project: Vue3 Diary App

Tech Stack

Frontend

- Framework: Vue 3 (Vite)
- API Style: Composition API (<script setup>)
- Styling: Bootstrap5
- State Management: Pinia (Composition API 사용)
- axios를 통한 백엔드 통신
- Icon Font: FontAwesome5

Backend

- Framework: json-server@v1.0.0-beta.15
- 데이터 양식: json
- 데이터 파일명: db.json

Gemini.md

Instructions

- 모든 컴포넌트는 SFC 스타일로 작성해줘.
- 변수명은 camelCase, 파일명은 PascalCase를 사용해줘.
- json-server 엔드포인트: diary
- 일기 저장 항목
 - id : PK, 문자열
 - date : 읽기 해당 날짜(시간 제외)
 - weather: 날씨
 - title : 제목
 - content: 일기 내용
 - createdAt: 작성일, 시간

페이지 구성

- 반응형 웹으로 화면 구성
 - sm에서 메뉴 접기, 그 이상에서는 형상 유지
- 타이틀, 레이블, 버튼, 링크 등을 만들 때 어울리는 FontAwesome 5 아이콘을 붙여줘.
- 앱의 첫 페이지는 일기장 목록 페이지로 함
- 캘린더를 사용하여 일기 목록 표시
- 일기 쓰기/수정하기/일기보기는 개별 페이지로 이동하여 수행

✓ Proxy 서버 설정

○ 프롬프트

Corss Origin 문제를 해결하는 하기위해 Proxy 설정을 해줘.

- 앱에서 사용하는 url은 /api 접두어로 시작
- 타겟 서버는 localhost:3000
- 서버와 실제 통신하는 경우 /api 접두어 제거

vite.config.js

```
...
server: {
  proxy: {
    '/api': {
      target: 'http://localhost:3000',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/api/, ''),
    },
  },
},
resolve: {
  alias: {
    '@': fileURLToPath(new URL('./src', import.meta.url))
  },
},
})
```

✓ api/diaryApi 모듈

○ 프롬프트

axios를 사용하여 /api/diary 엔드포인트로 rest 통신을 하는 모듈을 /src/api/diaryApi.js에 작성해줘.

src/api/diaryApi.js

```
import axios from 'axios';

const BASE_URL = '/api/diary';

/**
 * 일기 목록 전체를 가져옵니다.
 * @returns {Promise<Array>} 일기 목록 배열
 */
export const getDiaries = async () => {
  const response = await axios.get(BASE_URL);
  return response.data;
};

/**
 * 특정 ID의 일기를 가져옵니다.
 * @param {string} id 일기 PK
 * @returns {Promise<Object>} 일기 객체
 */
export const getDiaryById = async (id) => {
  const response = await axios.get(`${BASE_URL}/${id}`);
  return response.data;
};
```


src/api/diaryApi.js

```
/**
 * 새로운 일기를 작성합니다.
 * @param {Object} diary 일기 정보 (date, weather, title, content, createdAt)
 * @returns {Promise<Object>} 생성된 일기 객체
 */
export const createDiary = async (diary) => {
  const response = await axios.post(BASE_URL, diary);
  return response.data;
};

/**
 * 일기를 수정합니다.
 * @param {string} id 일기 PK
 * @param {Object} diary 수정할 일기 정보
 * @returns {Promise<Object>} 수정된 일기 객체
 */
export const updateDiary = async (id, diary) => {
  const response = await axios.put(`${BASE_URL}/${id}`, diary);
  return response.data;
};
```

src/api/diaryApi.js

```
/**
 * 일기를 삭제합니다.
 * @param {string} id 일기 PK
 * @returns {Promise<Object>} 삭제 결과
 */
export const deleteDiary = async (id) => {
  const response = await axios.delete(`${BASE_URL}/${id}`);
  return response.data;
};
```

✓ DiaryStore 모듈

○ 프롬프트

상태관리 store 모듈을 만들어줘.

src/stores/DiaryStore.js

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'
import * as diaryApi from '@api/diaryApi'

/**
 * 일기 관리를 위한 Pinia 스토어 (Composition API 스타일)
 */
export const useDiaryStore = defineStore('diary', () => {
  // === State (상태) ===
  const diaries = ref([])
  const loading = ref(false)
  const error = ref(null)

  // === Getters (계산된 상태) ===
  /**
   * 특정 날짜의 일기들을 필터링하여 반환하는 Getter
   * @param {string} dateString 'YYYY-MM-DD' 형식의 날짜 문자열
   */
  const getDiariesByDate = computed(() => (dateString) => {
    return diaries.value.filter((d) => d.date === dateString)
  })
})
```

src/stores/DiaryStore.js

```
// === Actions (함수) ===  
/**  
 * 서버에서 모든 일기 목록을 가져옵니다.  
 */  
const fetchDiaries = async () => {  
  loading.value = true  
  error.value = null  
  try {  
    const data = await diaryApi.getDiaries()  
    diaries.value = data  
  } catch (err) {  
    error.value = '일기 목록을 불러오는 데 실패했습니다.'  
    console.error(err)  
  } finally {  
    loading.value = false  
  }  
}
```

src/stores/DiaryStore.js

```
/**
 * 특정 ID의 일기를 서버에서 상세 조회합니다.
 * @param {string} id 일기 PK
 */
const fetchDiaryById = async (id) => {
  loading.value = true
  error.value = null
  try {
    return await diaryApi.getDiaryById(id)
  } catch (err) {
    error.value = '일기를 불러오는 데 실패했습니다.'
    console.error(err)
    throw err
  } finally {
    loading.value = false
  }
}
```

src/stores/DiaryStore.js

```
/**
 * 새로운 일기를 서버에 저장하고 로컬 상태를 업데이트합니다.
 * @param {Object} diary 일기 정보 객체
 */
const addDiary = async (diary) => {
  try {
    const newDiary = await diaryApi.createDiary(diary)
    diaries.value.push(newDiary)
    return newDiary
  } catch (err) {
    console.error('일기 저장 실패:', err)
    throw err
  }
}
```

src/stores/DiaryStore.js

```
/**
 * 기존 일기를 수정하고 로컬 상태를 업데이트합니다.
 * @param {string} id 일기 PK
 * @param {Object} diary 수정할 일기 정보
 */
const updateDiary = async (id, diary) => {
  try {
    const updatedDiary = await diaryApi.updateDiary(id, diary)
    const index = diaries.value.findIndex((d) => d.id === id)
    if (index !== -1) {
      diaries.value[index] = updatedDiary
    }
    return updatedDiary
  } catch (err) {
    console.error('일기 수정 실패:', err)
    throw err
  }
}
```


src/stores/DiaryStore.js

```
/**
 * 일기를 삭제하고 로컬 상태에서 제거합니다.
 * @param {string} id 일기 PK
 */
const deleteDiary = async (id) => {
  try {
    await diaryApi.deleteDiary(id)
    diaries.value = diaries.value.filter((d) => d.id !== id)
  } catch (err) {
    console.error('일기 삭제 실패:', err)
    throw err
  }
}
```

src/stores/DiaryStore.js

```
// 외부에 노출할 변수와 함수들 반환
return {
  // State
  diaries,
  loading,
  error,
  // Getters
  getDiariesByDate,
  // Actions
  fetchDiaries,
  fetchDiaryById,
  addDiary,
  updateDiary,
  deleteDiary
}
})
```

✓ router 모듈

○ 프롬프트

지시 사항에 맞게 라우트 테이블을 만들어줘. 각 페이지는 일단 제목만 넣어줘.

App.vue를 수정해서 전체적인 레이아웃을 잡아줘. 따뜻한 느낌의 파스텔 톤 테마로 Bootstrap5를 적용해줘.

src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import DiaryList from '../views/DiaryList.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'diary-list',
      component: DiaryList,
    },
    {
      path: '/diary/write',
      name: 'diary-write',
      component: () => import('../views/DiaryWrite.vue'),
    },
    {
      path: '/diary/:id',
      name: 'diary-detail',
      component: () => import('../views/DiaryDetail.vue'),
    },
  ],
})
```

views

- ▼ AboutView.vue
- ▼ DiaryDetail.vue
- ▼ DiaryEdit.vue
- ▼ DiaryList.vue
- ▼ DiaryWrite.vue
- ▼ HomeView.vue

src/components/DiaryHeader.vue

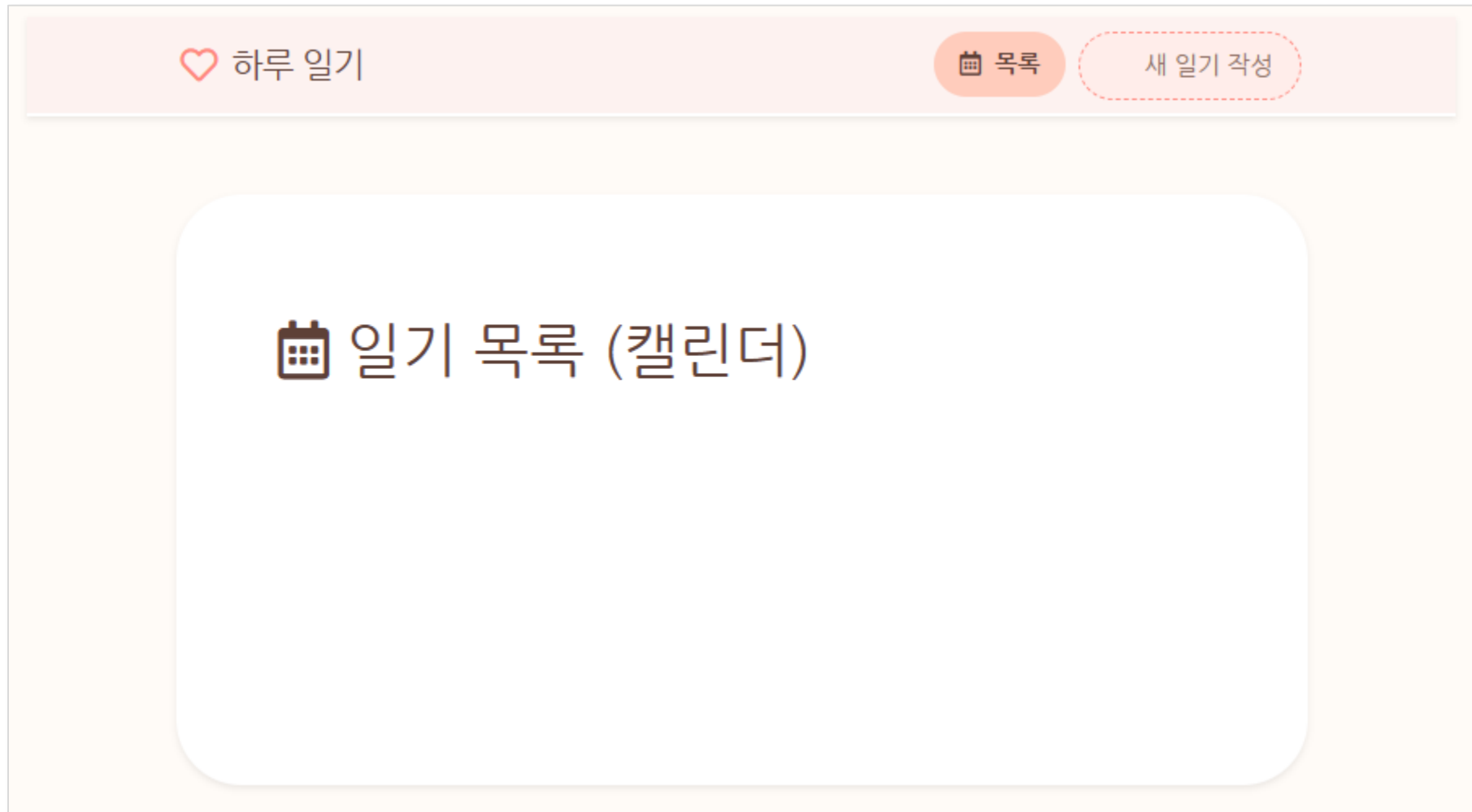
```
<script setup>
defineProps({
  title: String,
  subtitle: String
})
</script>

<template>
  <div class="text-center mb-5">
    <h2 class="fw-bold text-warm-dark mb-2">
      <i class="fas fa-calendar-heart text-pastel-red me-2"></i>{{ title }}
    </h2>
    <p class="text-warm-muted">{{ subtitle }}</p>
  </div>
</template>
```

src/router/index.js

```
{
  path: '/diary/edit/:id',
  name: 'diary-edit',
  component: () => import('../views/DiaryEdit.vue'),
},
],
})

export default router
```



✓ App.vue 수정

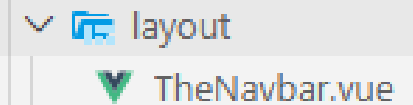
○ 프롬프트

- App.vue에서 메뉴 구성을 별도의 컴포넌트로 구현해서 다시 작성해줘.

✦ App.vue의 메뉴(내비게이션 바) 구성을 별도의 컴포넌트로 분리하여 구조를 개선했습니다.

주요 변경 사항:

1. TheNavbar.vue 생성: 기존의 복잡했던 내비게이션 바 코드를 `src/components/layout/TheNavbar.vue`로 옮겼습니다. 메뉴와 관련된 스타일도 `scoped`로 분리하여 관리하기 편리하게 만들었습니다.
2. App.vue 단순화: TheNavbar 컴포넌트를 임포트하여 사용하여 템플릿 구조가 훨씬 명확해졌습니다.
3. 트랜지션 추가: 페이지 전환 시 부드러운 효과를 주기 위해 RouterView에 `<transition>`(fade 효과)을 적용했습니다.
4. 전역 변수 유지: `--pastel-nav` 등 다른 컴포넌트에서도 공통으로 사용할 파스텔 톤 색상 변수들은 App.vue의 전역 `:root`에 남겨두어 일관된 디자인 시스템을 유지했습니다.



src/App.vue

```
<script setup>
import { RouterView } from 'vue-router'
import TheNavbar from '@components/layout/TheNavbar.vue'
</script>

<template>
  <div class="app-wrapper min-vh-100 d-flex flex-column">
    <header>
      <!-- 분리한 내비게이션 바 컴포넌트 -->
      <TheNavbar />
    </header>

    <main class="flex-grow-1 container my-5">
      <div class="row justify-content-center">
        <div class="col-12 col-lg-10 col-xl-8">
          <div class="diary-container bg-white p-4 p-md-5 rounded-5 shadow-sm border border-white border-3">
            <RouterView v-slot="{ Component }">
              <transition name="fade" mode="out-in">
                <component :is="Component" />
              </transition>
            </RouterView>
          </div>
        </div>
      </div>
    </main>
  </div>
</template>
```

src/App.vue

```
<footer class="footer text-center py-5">
  <div class="container">
    <div class="footer-divider mb-4 mx-auto"></div>
    <p class="text-warm-muted mb-0">
      <i class="fas fa-cloud-sun me-2"></i>
      당신의 소중한 하루를 따뜻하게 기록해 보세요.
    </p>
    <small class="text-warm-light d-block mt-2">&copy; 2026 Haru Diary. All rights reserved.</small>
  </div>
</footer>
</div>
</template>

<style>
...
</style>
```

✓ 일기장 목록보기

○ 프론트

일기장 목록 보기에 어울리는 캘린더 컴포넌트를 추천해줘.

→ V-Calendar 추천됨

V-Calendar를 이용하여 일기장 목록 페이지를 만들어줘. 데이터는 서버의 데이터를 받아서 연동해줘.

+ 먼저 V-Calendar를 설치하고, 서버 데이터를 연동하기 위해 DiaryStore.js와 diaryApi.js의 구성을 확인한 후 목록 페이지를 구현하겠습니다.

1. V-Calendar 설치 및 전역 설정

V-Calendar를 설치합니다.

Action Required

? Shell npm install v-calendar@next @popperjs/core [current working directory D:\2026_kb_...

npm install v-calendar@next @popperjs/core

Allow execution of: 'npm'?

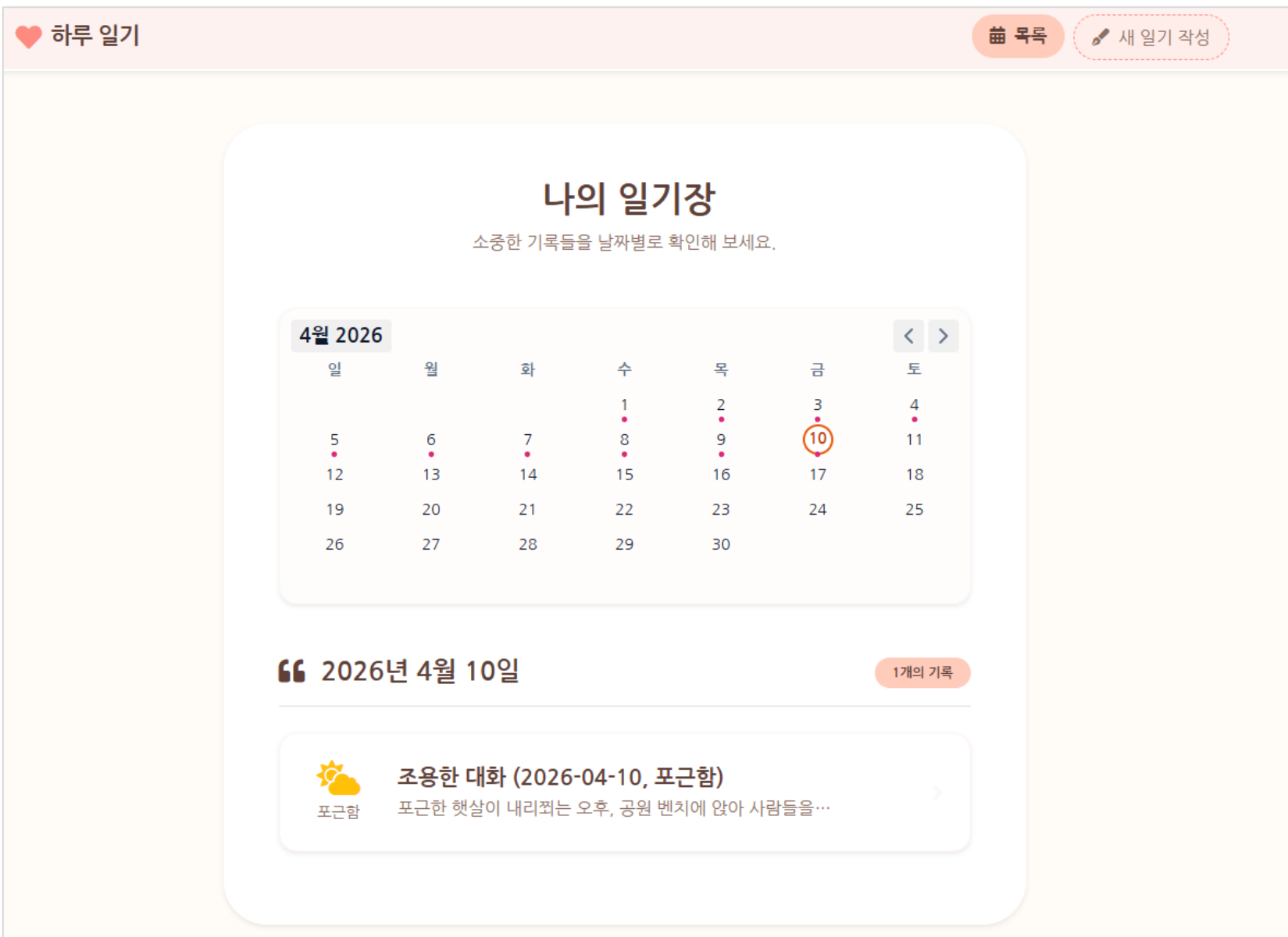
1. Allow once
- 2. Allow for this session
3. No, suggest changes (esc)

✓ 일기장 목록보기

○ 수정 프롬프트

캘린더에 오늘의 날짜에는 특수한 표시 해줘.

캘린더의 날짜를 클릭한 경우, 해당 날짜의 일기 목록이 나오도록 바꿔줘.



✓ 일기장 목록보기

○ 수정 프롬프트

DiaryList.vue의 내용을 항목별로 컴포넌트화해서 다시 작성해줘.

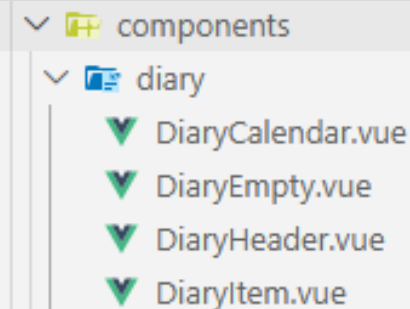
+ DiaryList.vue의 내용을 항목별로 컴포넌트화하여 재작성하였습니다.

컴포넌트 구조:

1. **DiaryHeader.vue**: 제목과 설명을 관리하며, 다른 페이지에서도 재사용 가능한 형태입니다.
2. **DiaryCalendar.vue**: V-Calendar 설정과 일기 데이터 점(Dot) 표시 로직을 캡슐화했습니다.
3. **DiaryItem.vue**: 개별 일기 카드를 담당하며 날씨 아이콘 매핑 로직을 포함합니다.
4. **DiaryEmpty.vue**: 작성된 일기가 없을 때의 뷰를 독립적으로 관리합니다.

개선된 점:

- 코드 가독성: **DiaryList.vue**의 코드가 200줄 이상에서 약 100줄 정도로 줄어들어 전체 흐름을 파악하기 매우 쉬워졌습니다.
- 유지보수성: 날씨 아이콘을 바꾸고 싶다면 **DiaryItem.vue**만, 캘린더 스타일을 바꾸고 싶다면 **DiaryCalendar.vue**만 수정하면 됩니다.
- 재사용성: 특히 헤더나 일기 항목 카드는 다른 화면에서도 쉽게 불러와 쓸 수 있습니다.



src/components/DiaryHeader.vue

```
<script setup>
defineProps({
  title: String,
  subtitle: String
})
</script>

<template>
  <div class="text-center mb-5">
    <h2 class="fw-bold text-warm-dark mb-2">
      <i class="fas fa-calendar-heart text-pastel-red me-2"></i>{{ title }}
    </h2>
    <p class="text-warm-muted">{{ subtitle }}</p>
  </div>
</template>
```

src/components/diary/DiaryItem.vue

```
<script setup>
const props = defineProps({
  diary: Object
})

const emit = defineEmits(['click'])

const getWeatherIcon = (weather) => {
  const icons = {
    '맑음': 'fa-sun text-warning',
    '흐림': 'fa-cloud text-secondary',
    '비': 'fa-cloud-showers-heavy text-primary',
    '눈': 'fa-snowflake text-info',
    '안개': 'fa-smog text-muted',
    '미세먼지 많음': 'fa-dust text-secondary',
    '포근함': 'fa-cloud-sun text-warning',
    '바람 불고 쌀쌀함': 'fa-wind text-info'
  }
  return icons[weather] || 'fa-smile text-pastel-red'
}
</script>
```


src/components/diary/DiaryItem.vue

```
<template>
  <div
    class="diary-card card border-0 shadow-sm mb-3 rounded-4 hover-lift cursor-pointer"
    @click="emit('click', diary.id)"
  >
    <div class="card-body p-4 d-flex align-items-center">
      <div class="weather-icon-box me-4 text-center">
        <i :class="['fas', getWeatherIcon(diary.weather), 'fa-2x']"></i>
        <div class="small text-warm-muted mt-1">{{ diary.weather }}</div>
      </div>
      <div class="flex-grow-1">
        <h5 class="card-title fw-bold mb-1 text-warm-dark">{{ diary.title }}</h5>
        <p class="card-text text-truncate text-warm-muted" style="max-width: 400px;">
          {{ diary.content }}
        </p>
      </div>
      <div class="text-end ms-3">
        <i class="fas fa-chevron-right text-light"></i>
      </div>
    </div>
  </div>
</template>

<style scoped>
...
</style>
```

src/components/diary/DiaryEmpty.vue

```
<script setup>
import { RouterLink } from 'vue-router'
</script>

<template>
  <div class="empty-state text-center py-5 rounded-4 bg-light border border-dashed">
    <i class="fas fa-feather-alt fa-3x text-light mb-3"></i>
    <p class="text-warm-muted mb-4">오늘의 기록이 아직 없네요.</p>
    <RouterLink :to="{ name: 'diary-write' }" class="btn btn-pastel-red rounded-pill px-4 py-2 shadow-sm text-white">
      <i class="fas fa-pen-fancy me-2"></i>첫 일기 쓰기
    </RouterLink>
  </div>
</template>

<style scoped>
.btn-pastel-red {
  background-color: #ff8a80;
  border: none;
}

.btn-pastel-red:hover {
  background-color: #ff5252;
}
...
```

src/components/diary/DiaryCalendar.vue

```
<script setup>
import { computed } from 'vue'

const props = defineProps({
  modelValue: Date,
  diaries: Array
})

const emit = defineEmits(['update:modelValue', 'dayclick'])

const selectedDate = computed({
  get: () => props.modelValue,
  set: (val) => emit('update:modelValue', val)
})
```

src/components/diary/DiaryCalendar.vue

```
const attributes = computed(() => {
  const attrs = [
    {
      key: 'today',
      highlight: {
        color: 'orange',
        fillMode: 'solid',
        contentClass: 'fw-bold text-white today-date',
      },
      dates: new Date(),
      popover: { label: '오늘 📅', visibility: 'hover' }
    }
  ]

  props.diaries.forEach(diary => {
    attrs.push({
      key: `diary-${diary.id}`,
      dot: { color: 'pink' },
      dates: new Date(diary.date),
      popover: { label: diary.title, visibility: 'hover' }
    })
  })

  return attrs
})
```

src/components/diary/DiaryCalendar.vue

```
const onDayClick = (day) => {
  emit('dayclick', day)
}
</script>

<template>
  <div class="calendar-wrapper mb-5 shadow-sm rounded-4 overflow-hidden border border-light">
    <VCalendar
      v-model="selectedDate"
      :attributes="attributes"
      expanded
      borderless
      transparent
      locale="ko"
      color="orange"
      title-position="left"
      @dayclick="onDayClick"
    />
  </div>
</template>

<style scoped>
...
```

src/viewss/DiaryList.vue

```
<script setup>
import { ref, onMounted, computed } from 'vue'
import { useRouter } from 'vue-router'
import { useDiaryStore } from '@stores/DiaryStore'

// 컴포넌트 임포트
import DiaryHeader from '@components/diary/DiaryHeader.vue'
import DiaryCalendar from '@components/diary/DiaryCalendar.vue'
import DiaryItem from '@components/diary/DiaryItem.vue'
import DiaryEmpty from '@components/diary/DiaryEmpty.vue'

const diaryStore = useDiaryStore()
const router = useRouter()

const selectedDate = ref(new Date())
const diaryListSection = ref(null)

onMounted(async () => {
  await diaryStore.fetchDiaries()
})
```

src/viewss/DiaryList.vue

```
// 날짜 클릭 시 처리
const handleDayClick = (day) => {
  selectedDate.value = day.date
  if (diaryListSection.value) {
    diaryListSection.value.scrollIntoView({ behavior: 'smooth', block: 'start' })
  }
}

// 선택된 날짜의 일기 필터링
const selectedDiaries = computed(() => {
  if (!selectedDate.value) return []
  const dateString = selectedDate.value.toISOString().split('T')[0]
  return diaryStore.getDiariesByDate(dateString)
})

const goDetail = (id) => {
  router.push({ name: 'diary-detail', params: { id } })
}

const formattedSelectedDate = computed(() => {
  return selectedDate.value.toLocaleDateString('ko-KR', {
    year: 'numeric', month: 'long', day: 'numeric'
  })
})
</script>
```

src/viewss/DiaryList.vue

```
<template>
  <div class="diary-list-view">
    <!-- 1. 헤더 영역 -->
    <DiaryHeader
      title="나의 일기장"
      subtitle="날짜를 클릭하여 그날의 기록을 확인하세요."
    />

    <!-- 2. 캘린더 영역 -->
    <DiaryCalendar
      v-model="selectedDate"
      :diaries="diaryStore.diaries"
      @dayclick="handleDayClick"
    />

    <!-- 3. 일기 목록 영역 -->
    <div ref="diaryListSection" class="diary-entries-section mt-4 pt-2">
      <div class="d-flex justify-content-between align-items-center border-bottom pb-3 mb-4">
        <h4 class="fw-bold m-0 text-warm-dark">
          <i class="fas fa-quote-left text-pastel-orange me-2"></i>
          {{ formattedSelectedDate }}
        </h4>
        <span class="badge rounded-pill bg-pastel-orange text-warm-dark px-3 py-2">
          {{ selectedDiaries.length }}개의 기록
        </span>
      </div>
    </div>
  </div>
```


src/viewss/DiaryList.vue

```
<!-- 로딩 중 -->
<div v-if="diaryStore.loading" class="text-center py-5">
  <div class="spinner-border text-pastel-red" role="status"></div>
</div>

<!-- 일기 목록 표시 -->
<template v-else-if="selectedDiaries.length > 0">
  <DiaryItem
    v-for="diary in selectedDiaries"
    :key="diary.id"
    :diary="diary"
    @click="goDetail"
  />
</template>

<!-- 일기 없음 -->
<DiaryEmpty v-else />
</div>
</div>
</template>

<style scoped>
...
```

✓ 일기 작성 페이지

○ 프롬프트

DiaryWrite 페이지를 작성해줘. 작성한 일기가 실제 서버에 저장되도록 해줘.

+ 일기 쓰기 페이지(DiaryWrite.vue) 구현을 완료했습니다.

주요 기능:

- 반응형 폼: 날짜 선택, 날씨 선택(커스텀 라디오 버튼), 제목, 내용 입력란을 포함합니다.
- 날씨 아이콘 선택: '맑음', '흐림' 등 각 날씨에 맞는 FontAwesome 아이콘을 버튼 형태로 제공하여 직관적인 선택이 가능합니다.
- 실제 데이터 저장: DiaryStore를 통해 json-server의 diary 엔드포인트로 POST 요청을 보내 일기를 저장합니다.
- 감성적인 디자인: 따뜻한 브라운 컬러의 레이블과 파스텔 레드 색상의 저장 버튼을 사용했습니다.
- 사용자 경험(UX):
 - 저장 중 버튼 비활성화 및 로딩 스피너 표시
 - 작성 취소 시 확인 창 표시
 - 저장 완료 후 목록 페이지로 자동 이동 및 알림 표시

새 일기 쓰기

오늘 하루는 어땠나요? 마음의 조각을 남겨보세요.

📅 날짜

2026-04-10

🌤️ 오늘 날씨

☀️ 맑음

☁️ 흐림

🌧️ 비

❄️ 눈

🌪️ 포근함

💧 쌀쌀함

H 제목

제목을 입력하세요...

📝 내용

여기에 오늘의 이야기를 들려주세요...

✕ 취소

♥️ 저장하기

src/views/DiaryWrite.vue

```
<script setup>
import { ref, reactive } from 'vue'
import { useRouter } from 'vue-router'
import { useDiaryStore } from '@stores/DiaryStore'
import DiaryHeader from '@components/diary/DiaryHeader.vue'

const diaryStore = useDiaryStore()
const router = useRouter()

// 폼 데이터 초기화
const form = reactive({
  date: new Date().toISOString().split('T')[0], // 오늘 날짜 기본값
  weather: '맑음',
  title: '',
  content: ''
})

// 날씨 옵션 정의
const weatherOptions = [
  { label: '맑음', value: '맑음', icon: 'fa-sun text-warning' },
  { label: '흐림', value: '흐림', icon: 'fa-cloud text-secondary' },
  { label: '비', value: '비', icon: 'fa-cloud-showers-heavy text-primary' },
  { label: '눈', value: '눈', icon: 'fa-snowflake text-info' },
  { label: '포근함', value: '포근함', icon: 'fa-cloud-sun text-warning' },
  { label: '쌀쌀함', value: '바람 불고 쌀쌀함', icon: 'fa-wind text-info' }
]
```

src/views/DiaryWrite.vue

```
const isSubmitting = ref(false)

// 일기 저장
const handleSubmit = async () => {
  if (!form.title.trim() || !form.content.trim()) {
    alert('제목과 내용을 모두 입력해 주세요! ✍️ ')
    return
  }

  isSubmitting.value = true
  try {
    const newEntry = {
      ...form,
      id: Date.now().toString(), // 임시 ID 생성 (json-server가 생성해주기도 하지만 명시적 지정)
      createdAt: new Date().toISOString()
    }

    await diaryStore.addDiary(newEntry)
    alert('일기가 소중한게 저장되었습니다! ♥ ')
    router.push({ name: 'diary-list' })
  } catch (error) {
    alert('저장 중 오류가 발생했습니다. 다시 시도해 주세요.')
    console.error(error)
  } finally {
    isSubmitting.value = false
  }
}
```

src/views/DiaryWrite.vue

```
const goBack = () => {  
  if (confirm('작성 중인 내용이 사라집니다. 취소할까요?')) {  
    router.back()  
  }  
}  
</script>
```

src/views/DiaryWrite.vue

```
<template>
  <div class="diary-write-view">
    <DiaryHeader
      title="새 일기 쓰기"
      subtitle="오늘 하루는 어땠나요? 마음의 조각을 남겨보세요."
    />

    <form @submit.prevent="handleSubmit" class="diary-form mt-4">
      <!-- 날짜 및 날씨 선택 -->
      <div class="row mb-4">
        <div class="col-md-6 mb-3 mb-md-0">
          <label class="form-label fw-bold text-warm-dark">
            <i class="fas fa-calendar-alt me-1 text-pastel-red"></i> 날짜
          </label>
          <input
            type="date"
            v-model="form.date"
            class="form-control rounded-3 border-warm-light p-2 shadow-none"
            required
          >
        </div>
      </div>
    </form>
  </div>
```

src/views/DiaryWrite.vue

```
<div class="col-md-6">
  <label class="form-label fw-bold text-warm-dark">
    <i class="fas fa-thermometer-half me-1 text-pastel-red"></i> 오늘 날씨
  </label>
  <div class="weather-selector d-flex flex-wrap gap-2">
    <div v-for="opt in weatherOptions" :key="opt.value" class="weather-option">
      <input
        type="radio"
        :id="opt.value"
        :value="opt.value"
        v-model="form.weather"
        class="btn-check"
      >
      <label
        :for="opt.value"
        class="btn btn-outline-pastel-warm rounded-pill px-3 py-1 d-flex align-items-center"
      >
        <i :class="['fas', opt.icon, 'me-1']"></i>
        <span class="small">{{ opt.label }}</span>
      </label>
    </div>
  </div>
</div>
```

src/views/DiaryWrite.vue

```
<!-- 제목 입력 -->
<div class="mb-4">
  <label class="form-label fw-bold text-warm-dark">
    <i class="fas fa-heading me-1 text-pastel-red"></i> 제목
  </label>
  <input
    type="text"
    v-model="form.title"
    placeholder="제목을 입력하세요..."
    class="form-control rounded-3 border-warm-light p-3 shadow-none title-input"
    maxlength="50"
    required
  >
</div>
```


src/views/DiaryWrite.vue

```
<!-- 내용 입력 -->
<div class="mb-5">
  <label class="form-label fw-bold text-warm-dark">
    <i class="fas fa-pen-nib me-1 text-pastel-red"></i> 내용
  </label>
  <textarea
    v-model="form.content"
    placeholder="여기에 오늘의 이야기를 들려주세요..."
    class="form-control rounded-4 border-warm-light p-4 shadow-none content-textarea"
    rows="12"
    required
  ></textarea>
</div>
```

src/views/DiaryWrite.vue

```
<!-- 하단 버튼 -->
<div class="d-flex justify-content-center gap-3 mb-4">
  <button
    type="button"
    @click="goBack"
    class="btn btn-outline-secondary rounded-pill px-5 py-2 shadow-sm"
    :disabled="isSubmitting"
  >
    <i class="fas fa-times me-2"></i>취소
</button>
<button
  type="submit"
  class="btn btn-pastel-red rounded-pill px-5 py-2 shadow-sm text-white fw-bold"
  :disabled="isSubmitting"
  >
    <span v-if="isSubmitting" class="spinner-border spinner-border-sm me-2"></span>
    <i v-else class="fas fa-heart me-2"></i>저장하기
</button>
</div>
</form>
</div>
</template>

<style scoped>
...
```

✓ 일기 보기 페이지

○ 프롬프트

DiaryDetail 페이지를 작성해줘.

→ **DiaryEdit.vue** 페이지를 구현하겠습니다. **DiaryWrite.vue**와 유사한 폼 구조를 유지하되, 기존 데이터를 불러와서 수정할 수 있도록 로직을 구성하겠습니다.

DiaryEdit.vue 구현

구현 포인트:

- 데이터 로드: 페이지 진입 시 해당 ID의 일기 데이터를 가져와 폼에 채웁니다.
- 날짜 선택: 기존에 선택된 날짜를 시각적으로 표시합니다.
- 수정 로직: **DiaryStore**의 **updateDiary** 액션을 호출하여 변경된 내용을 저장합니다.
- 디자인 일관성: 따뜻한 파스텔 톤 디자인과 컴포넌트(**DiaryHeader**)를 재사용합니다.

🌸 포근함

조용한 대화 (2026-04-10, 포근함)

📅 2026년 4월 10일 금요일

포근한 햇살이 내리쬐는 오후, 공원 벤치에 앉아 사람들을 구경했다. 각자의 삶을 바쁘게 살아가는 그들의 뒷모습에서 묘한 동질감을 느꼈다. 우리 모두는 각자의 방식으로 최선을 다해 사랑하고 아파하며 살아가고 있는 것이다.

때로는 침묵이 백 마디 말보다 더 큰 위로가 될 때가 있다. 누군가의 곁에서 그저 묵묵히 자리를 지켜주는 것만으로도 충분한 사랑일 수 있다. 책장의 오래된 책 냄새가 마음을 차분하게 가라앉혀 준다. 오래된 문장들은 시간이 흘러도 여전히 그 자리에 머물러 나를 기다려준다.

아무도 없는 방 안에 홀로 앉아 시계 소리를 듣고 있으면, 내 안의 목소리가 더 선명하게 들려온다. 평소에는 소음에 가려져 듣지 못했던 진심들이 하나둘 떠오를 때면 조금은 아릿한 기분이 들기도 한다. 하지만 이런 시간들이 있어야 비로소 나는 나로서 존재할 수 있다. 고독은 외로움이 아니라 나를 만나는 시간이다.

오늘 날씨가 포근함여서 그랬을까. 2026-04-10의 공기는 유난히 기억에 남을 것 같다. 기록을 남기며 하루를 마무리하는 이 평온함이 참 좋다. 내일은 또 다른 계절의 향기가 나를 맞이해 주기를 기대해 본다.

🕒 기록 시각: 2026. 4. 10. 오후 9:41:10

☰ 목록으로

✎ 수정하기

🗑 삭제

src/views/DiaryDetail.vue

```
<script setup>
import { ref, onMounted, computed } from 'vue';
import { useRoute, useRouter } from 'vue-router';
import { useDiaryStore } from '@stores/DiaryStore';

const route = useRoute();
const router = useRouter();
const diaryStore = useDiaryStore();

const diary = ref(null);
const diaryId = route.params.id;

onMounted(async () => {
  try {
    diary.value = await diaryStore.fetchDiaryById(diaryId);
  } catch (error) {
    console.log(error);
    alert('일기를 찾을 수 없습니다.');
    router.push({ name: 'diary-list' });
  }
});
```

src/views/DiaryDetail.vue

```
// 날씨 아이콘 매핑
const getWeatherIcon = (weather) => {
  const icons = {
    맑음: 'fa-sun text-warning',
    흐림: 'fa-cloud text-secondary',
    비: 'fa-cloud-showers-heavy text-primary',
    눈: 'fa-snowflake text-info',
    안개: 'fa-smog text-muted',
    '미세먼지 많음': 'fa-dust text-secondary',
    포근함: 'fa-cloud-sun text-warning',
    '바람 불고 쌀쌀함': 'fa-wind text-info',
  };
  return icons[weather] || 'fa-smile text-pastel-red';
};

// 날짜 포맷
const formattedDate = computed(() => {
  if (!diary.value) return '';
  return new Date(diary.value.date).toLocaleDateString('ko-KR', {
    year: 'numeric',
    month: 'long',
    day: 'numeric',
    weekday: 'long',
  });
});
```

src/views/DiaryDetail.vue

```
const goList = () => router.push({ name: 'diary-list' });
const goEdit = () => router.push({ name: 'diary-edit', params: { id: diaryId } });

const handleDelete = async () => {
  if (confirm('정말로 이 소중한 일기를 삭제하시겠어요? 😞')) {
    try {
      await diaryStore.deleteDiary(diaryId);
      alert('일기가 삭제되었습니다.');
      router.push({ name: 'diary-list' });
    } catch (error) {
      alert('삭제 중 오류가 발생했습니다.');
    }
  }
};
</script>
```

src/views/DiaryDetail.vue

```
<template>
  <div class="diary-detail-view">
    <!-- 로딩 중 -->
    <div v-if="diaryStore.loading" class="text-center py-5">
      <div class="spinner-border text-pastel-red" role="status"></div>
    </div>

    <template v-else-if="diary">
      <!-- 상단 헤더 -->
      <div class="diary-detail-header text-center mb-5">
        <div class="mb-3">
          <span class="badge rounded-pill bg-pastel-orange text-warm-dark px-3 py-2 shadow-sm">
            <i :class="['fas', getWeatherIcon(diary.weather), 'me-2']"></i>
            {{ diary.weather }}
          </span>
        </div>
        <h2 class="fw-bold text-warm-dark mb-3">{{ diary.title }}</h2>
        <p class="text-warm-muted small mb-0"><i class="fas fa-calendar-alt me-1"></i> {{ formattedDate }}</p>
      </div>
    </template>
  </div>
```

src/views/DiaryDetail.vue

```
<!-- 본문 영역 -->
<div class="diary-content-body p-4 p-md-5 rounded-4 bg-light shadow-inner mb-5">
  <div class="content-text text-warm-dark mb-4">
    {{ diary.content }}
  </div>
  <div class="text-end border-top pt-4">
    <small class="text-warm-light italic">
      <i class="fas fa-clock me-1"></i> 기록 시각: {{ new Date(diary.createdAt).toLocaleString() }}
    </small>
  </div>
</div>
```


src/views/DiaryDetail.vue

```
<!-- 하단 버튼 영역 -->
<div class="d-flex flex-wrap justify-content-between gap-3 mb-4">
  <button @click="goList" class="btn btn-outline-warm rounded-pill px-4">
    <i class="fas fa-list me-2"></i>목록으로
  </button>
  <div class="d-flex gap-2">
    <button @click="goEdit" class="btn btn-pastel-orange rounded-pill px-4 text-warm-dark fw-bold">
      <i class="fas fa-edit me-2"></i>수정하기
    </button>
    <button @click="handleDelete" class="btn btn-outline-danger border-0 rounded-pill px-3">
      <i class="fas fa-trash-alt me-2"></i>삭제
    </button>
  </div>
</div>
</template>
</div>
</template>

<style scoped>
...
```

✓ 일기 수정 페이지

○ 프롬프트

DiaryEdit 페이지를 작성해줘.

✦ 일기 수정 페이지(DiaryEdit.vue) 구현을 완료했습니다.

주요 기능:

- 기존 데이터 연동: 페이지 진입 시 해당 일기의 정보를 서버에서 불러와 폼에 자동으로 채워줍니다.
- 날씨 선택 유지: 기존에 선택했던 날씨가 활성화된 상태로 표시되어 쉽게 변경할 수 있습니다.
- 실시간 데이터 업데이트: 수정 완료 후 `updateDiary` 액션을 통해 서버와 로컬 스토어를 동시에 업데이트하며, 상세 페이지로 돌아갑니다.
- 일관된 UX: 일기 쓰기 페이지와 동일한 입력 방식을 제공하여 사용자가 혼동 없이 수정할 수 있도록 했습니다.

일기 수정하기

기록된 소중한 추억을 다듬어 보세요.

날짜

2026-04-10

오늘 날씨

맑음

흐림

비

눈

포근함

쌀쌀함

H 제목

조용한 대화 (2026-04-10, 포근함)

내용

포근한 햇살이 내리쬐는 오후, 공원 벤치에 앉아 사람들을 구경했다. 각자의 삶을 바쁘게 살아가는 그들의 뒷모습에서 묘한 동질감을 느꼈다. 우리 모두는 각자의 방식으로 최선을 다해 사랑하고 아파하며 살아가고 있는 것이다.

때로는 침묵이 백 마디 말보다 더 큰 위트가 될 때가 있다. 누군가의 곁에서 그저 묵묵히 자리를 지켜주는 것만으로도 충분한 사랑일 수 있다. 책장의 오래된 책 냄새가 마음을 차분하게 가라앉혀 준다. 오래된 문장들은 시간이 흘러도 여전히 그 자리에 머물러 나를 기다려준다.

아무도 없는 방 안에 홀로 앉아 시계 소리를 듣고 있으면, 내 안의 목소리가 더 선명하게 들려온다. 평소에는 소음에 가려져 듣지 못했던 진심들이 하나둘 떠오를 때면 조금은 아릿한 기분이 들기도 한다. 하지만 이런 시간들이 있어야 비로소 나는 나로서 존재할 수 있다. 고독은 외로움이 아니라 나를 만나는 시간이다.

✕ 취소

✓ 수정 완료

src/views/DiaryEdit.vue

```
<script setup>
import { ref, onMounted, computed } from 'vue';
import { useRoute, useRouter } from 'vue-router';
import { useDiaryStore } from '@stores/DiaryStore';

const route = useRoute();
const router = useRouter();
const diaryStore = useDiaryStore();

const diary = ref(null);
const diaryId = route.params.id;

onMounted(async () => {
  try {
    diary.value = await diaryStore.fetchDiaryById(diaryId);
  } catch (error) {
    console.log(error);
    alert('일기를 찾을 수 없습니다.');
    router.push({ name: 'diary-list' });
  }
});
```

src/views/DiaryEdit.vue

```
// 날씨 아이콘 매핑
const getWeatherIcon = (weather) => {
  const icons = {
    맑음: 'fa-sun text-warning',
    흐림: 'fa-cloud text-secondary',
    비: 'fa-cloud-showers-heavy text-primary',
    눈: 'fa-snowflake text-info',
    안개: 'fa-smog text-muted',
    '미세먼지 많음': 'fa-dust text-secondary',
    포근함: 'fa-cloud-sun text-warning',
    '바람 불고 쌀쌀함': 'fa-wind text-info',
  };
  return icons[weather] || 'fa-smile text-pastel-red';
};

// 날짜 포맷
const formattedDate = computed(() => {
  if (!diary.value) return '';
  return new Date(diary.value.date).toLocaleDateString('ko-KR', {
    year: 'numeric',
    month: 'long',
    day: 'numeric',
    weekday: 'long',
  });
});
```

src/views/DiaryEdit.vue

```
const goList = () => router.push({ name: 'diary-list' });
const goEdit = () => router.push({ name: 'diary-edit', params: { id: diaryId } });

const handleDelete = async () => {
  if (confirm('정말로 이 소중한 일기를 삭제하시겠어요? 😞')) {
    try {
      await diaryStore.deleteDiary(diaryId);
      alert('일기가 삭제되었습니다.');
      router.push({ name: 'diary-list' });
    } catch (error) {
      alert('삭제 중 오류가 발생했습니다.');
    }
  }
};
</script>
```

src/views/DiaryEdit.vue

```
<template>
  <div class="diary-detail-view">
    <!-- 로딩 중 -->
    <div v-if="diaryStore.loading" class="text-center py-5">
      <div class="spinner-border text-pastel-red" role="status"></div>
    </div>

    <template v-else-if="diary">
      <!-- 상단 헤더 -->
      <div class="diary-detail-header text-center mb-5">
        <div class="mb-3">
          <span class="badge rounded-pill bg-pastel-orange text-warm-dark px-3 py-2 shadow-sm">
            <i :class="['fas', getWeatherIcon(diary.weather), 'me-2']"></i>
            {{ diary.weather }}
          </span>
        </div>
        <h2 class="fw-bold text-warm-dark mb-3">{{ diary.title }}</h2>
        <p class="text-warm-muted small mb-0"><i class="fas fa-calendar-alt me-1"></i> {{ formattedDate }}</p>
      </div>
    </template>
  </div>
```

src/views/DiaryEdit.vue

```
<!-- 본문 영역 -->
<div class="diary-content-body p-4 p-md-5 rounded-4 bg-light shadow-inner mb-5">
  <div class="content-text text-warm-dark mb-4">
    {{ diary.content }}
  </div>
  <div class="text-end border-top pt-4">
    <small class="text-warm-light italic">
      <i class="fas fa-clock me-1"></i> 기록 시각: {{ new Date(diary.createdAt).toLocaleString() }}
    </small>
  </div>
</div>
```

src/views/DiaryEdit.vue

```
<!-- 하단 버튼 영역 -->
<div class="d-flex flex-wrap justify-content-between gap-3 mb-4">
  <button @click="goList" class="btn btn-outline-warm rounded-pill px-4">
    <i class="fas fa-list me-2"></i>목록으로
  </button>
  <div class="d-flex gap-2">
    <button @click="goEdit" class="btn btn-pastel-orange rounded-pill px-4 text-warm-dark fw-bold">
      <i class="fas fa-edit me-2"></i>수정하기
    </button>
    <button @click="handleDelete" class="btn btn-outline-danger border-0 rounded-pill px-3">
      <i class="fas fa-trash-alt me-2"></i>삭제
    </button>
  </div>
</div>
</template>
</div>
</template>

<style scoped>
...
```